

19.1-AI-ASS 2303A52249 A.VAMSHI KRISHNA

TASK-1

Prompt: Translate a simple Python class into Java using AI assistance.

Instructions:

Prompt AI to convert a Python class with attributes and methods into equivalent Java code.

Verify that constructors, method signatures, and data types are properly handled.

Run both versions (Python & Java) to compare expected outputs.

Expected Output: Equivalent working Java class translated from Python.

Code & Output:

The screenshot displays a Google Colab notebook titled "19.1-AI-ASS-2249". The notebook is divided into two main sections. The first section, titled "TASK-1", contains a Python class named "Person" with attributes "name" and "age", and methods "greet()", "get_older()", and "get_name()". The second section, titled "Python to Java Conversion", shows the AI-generated Java code. The Java code is a direct translation of the Python code, using "private String" for attributes, "public" for methods, and "this" for the instance. The output of the Java code is shown in the "Terminal" tab, which matches the output of the Python code. The AI assistant's response is also visible on the right side of the notebook, showing the prompt and the generated code.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        return f"Hello, my name is {self.name} and I am {self.age} years old."

    def get_older(self, years):
        self.age += years
        return f"I am now {self.age} years old."

# Example usage of the Python class
python_person = Person("Alice", 30)
print(python_person.greet())
print(python_person.get_older(5))
print(python_person.greet())

Hello, my name is Alice and I am 30 years old.
I am now 35 years old.
Hello, my name is Alice and I am 35 years old.
```

```
public class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String greet() {
        return "Hello, my name is " + this.name + " and I am " + this.age + " years old.";
    }

    public String getOlder(int years) {
        this.age += years;
        return "I am now " + this.age + " years old.";
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public static void main(String[] args) {
        Person javaPerson = new Person("Alice", 30);
        System.out.println(javaPerson.greet());
        System.out.println(javaPerson.getOlder(5));
        System.out.println(javaPerson.greet());
    }
}
```

Observation: Python to Java Conversion (Observation)

AI successfully translated Python class structures into Java by adapting dynamic typing into static typing.

Constructors, data types, and method signatures required careful verification. Minor adjustments were needed for syntax differences, but functionality remained consistent across both implementations.

TASK-2

Prompt: Convert a C++ function for factorial calculation into Python.

Instructions:

Ask AI to translate a recursive C++ factorial function into Python.

Validate correctness by testing with sample inputs.

Ensure Python version follows Pythonic conventions.

Expected Output: A correct Python implementation equivalent to the original C++ factorial function.

Code & Output:

The screenshot shows a Google Colab notebook titled "TASK-2" with the subtitle "C++ to Python Function Conversion". The notebook content includes a C++ recursive factorial function and a prompt to translate it into Python. The Python code is shown below the C++ code. A Gemini AI chat window on the right side of the notebook provides instructions and a confirmation to proceed.

```
long long factorial(int n) {
    if (n == 0) {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}

// Example usage in C++:
// int main() {
//     int num = 5;
//     std::cout << "Factorial of " << num << " is " << factorial(num) << std::endl; // Output: 120
//     return 0;
// }
```

Now, let's translate this C++ recursive factorial function into Python.

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)

# Validate correctness by testing with sample inputs
print(f"Factorial of 0 is: {factorial(0)}")
print(f"Factorial of 1 is: {factorial(1)}")
print(f"Factorial of 5 is: {factorial(5)}")
print(f"Factorial of 10 is: {factorial(10)}")
```

Task 2 - C++ to Python Function Conversion Task:
Convert a C++ function for factorial calculation into Python. Instructions:
• Ask AI to translate a recursive C++ factorial function into Python.
• Validate correctness by testing with sample inputs.
• Ensure Python version follows Pythonic conventions.
Expected Output: A correct Python implementation equivalent to the original C++ factorial function.

Okay, let's proceed

What can I help you b

+ Gemini 2.5 >

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Now, let's translate this C++ recursive factorial function into Python.

The screenshot shows a code editor with the Python implementation of the factorial function. The code is as follows:

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)

# Validate correctness by testing with sample inputs
print(f"Factorial of 0 is: {factorial(0)}")
print(f"Factorial of 1 is: {factorial(1)}")
print(f"Factorial of 5 is: {factorial(5)}")
print(f"Factorial of 10 is: {factorial(10)}")
```

The output of the code is:

```
Factorial of 0 is: 1
Factorial of 1 is: 1
Factorial of 5 is: 120
Factorial of 10 is: 3628800
```

Observation: C++ to Python Function Conversion (Observation)

The recursive factorial function was accurately converted into Python with simpler and more readable syntax. Python's concise structure reduced code complexity. Validation with test cases confirmed equivalent outputs, demonstrating effective cross-language translation.

TASK-3

Prompt: Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

Provide AI with a SQL query (e.g., `SELECT name, salary FROM employees WHERE salary > 50000`).

Ask AI to generate equivalent Pandas code.

Test the generated code on a sample DataFrame.

Expected Output: Equivalent Pandas query that retrieves the correct subset of data.

Code & Output:

The screenshot shows a Google Colab notebook titled "TASK-3" with the subtitle "SQL to Pandas Query Conversion". The notebook contains the following code and output:

```
import pandas as pd

# Create a sample Pandas DataFrame to simulate the 'employees' table
employees_df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank'],
    'age': [30, 24, 35, 40, 28, 55],
    'department': ['Sales', 'Marketing', 'Sales', 'IT', 'Sales', 'Marketing'],
    'salary': [60000, 45000, 75000, 90000, 52000, 48000]
})

print("Original DataFrame:")
display(employees_df)
```

The output of the above code is a DataFrame with the following data:

	name	age	department	salary
0	Alice	30	Sales	60000
1	Bob	24	Marketing	45000
2	Charlie	35	Sales	75000
3	David	40	IT	90000
4	Eve	28	Sales	52000
5	Frank	55	Marketing	48000

Next steps: [Generate code with employees_df](#) [New interactive sheet](#)

Now, let's translate the SQL query into equivalent Pandas code and display the results.

```
# Equivalent Pandas query
pandas_query_result = employees_df[
    (employees_df['salary'] > 50000) & (employees_df['department'] == 'Sales')
][['name', 'salary']]

print("Result of Pandas query (equivalent to SQL):")
display(pandas_query_result)
```

The output of the above code is a DataFrame with the following data:

	name	salary
0	Alice	60000
2	Charlie	75000
4	Eve	52000

Observation: SQL to Pandas Query (Observation)

AI effectively mapped SQL operations to Pandas DataFrame syntax. Filtering conditions and column selection were translated correctly using boolean indexing. The Pandas implementation produced the same results as the SQL query, ensuring data consistency.

TASK-4

Real-Time Application: Multi-Language Snippet Converter

Scenario: Students will choose a real-time use case where they need code in multiple languages.

Example: Converting a Python web scraping snippet into JavaScript for browser automation.

Translating a Java method into C# for enterprise applications.

Prompt:

AI to translate code between two chosen languages.

Test both implementations to ensure they produce the same results.

Document challenges faced during translation (e.g., syntax, libraries).

Code & Output:

The screenshot displays a Google Colab notebook titled "TASK-4: Multi-Language Snippet Converter: Python to JavaScript Fibonacci". The notebook is divided into three main sections:

- Python Implementation:** A recursive Fibonacci function in Python. The code is as follows:

```
def fibonacci_python(n):
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci_python(n - 1) + fibonacci_python(n - 2)

# Test the Python Implementation
print(f'Fibonacci (Python) of 0 is: {fibonacci_python(0)}')
print(f'Fibonacci (Python) of 1 is: {fibonacci_python(1)}')
print(f'Fibonacci (Python) of 5 is: {fibonacci_python(5)}')
print(f'Fibonacci (Python) of 10 is: {fibonacci_python(10)}')
```
- JavaScript Implementation:** The equivalent recursive Fibonacci function in JavaScript. The code is as follows:

```
function fibonacciJavaScript(n) {
    if (n <= 0) {
        return 0;
    } else if (n === 1) {
        return 1;
    } else {
        return fibonacciJavaScript(n - 1) + fibonacciJavaScript(n - 2);
    }
}
```
- Verification and Comparison:** A section titled "Verification and Comparison" that provides instructions on how to test the JavaScript code. It includes a list of steps: 1. Save the JavaScript code as 'fibonacci.js'. 2. Run the JavaScript code using Node.js. The notebook also shows the output of the JavaScript code, which matches the Python output: Fibonacci (Python) of 5 is: 5 and Fibonacci (Python) of 10 is: 55.

The notebook interface includes a sidebar with navigation icons, a top bar with the Colab logo and file name, and a bottom bar with a terminal and variables panel. The right sidebar shows a Gemini chat window with a prompt about converting a Python web scraping snippet into JavaScript for browser automation.

Observation: Multi-Language Snippet Converter (Observation)

Code translation between different languages was achieved with functional equivalence, but challenges arose due to differences in libraries, syntax, and runtime environments. Testing was essential to ensure accuracy. AI significantly reduced development effort while highlighting the need for manual optimization and debugging.